Alternative Assessment Tool (AAT) Report on

"Execution of Combinational Module and Sequential Module on the Cadence Tool"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Archit Jain **1BF24EC028**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

Table of Contents

1. Introduction and Objectives 3

2. Tools, Environment and Simulation Flow 3

3. Part A – Combinational Module: 4-bit Magnitude Comparator 4

3.1 Theory 4

3.2 Logic equations 4

3.3 Verilog RTL 5

3.4 Testbench 6

3.5 Execution on the Cadence tool 6

3.6 Results and verification 7

3.7 Observations 7

4. Part B – Sequential Module: 4-bit Ring Counter 8

4.1 Theory 8

4.2 Verilog RTL 9

4.3 Self-checking testbench 10

4.4 Execution on the Cadence tool 12

4.5 Results and verification 13

4.6 Observations 14

5. Notes on the Tool Logs 14

6. Summary and Conclusion 15

1. Introduction and Objectives

Hardware description languages (HDLs) allow digital systems to be described, simulated and verified long before any silicon or FPGA is involved. In this Alternative Assessment Tool (AAT), two Verilog designs – one **combinational** and one **sequential** – are written, compiled, elaborated and simulated using the **Cadence NC-Verilog (Incisive 15.20)** tool chain, and the resulting waveforms are inspected in SimVision.

Objectives

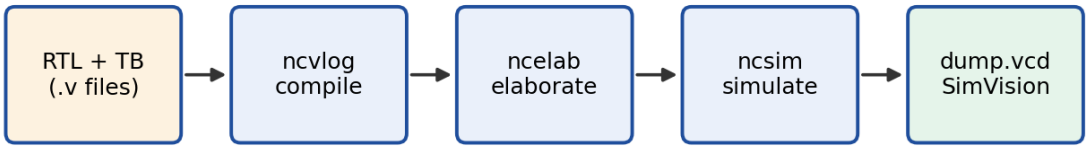
- To design a 4-bit cascadable magnitude comparator as an example of a purely combinational module.
- To design a parameterised N-bit ring counter with asynchronous reset as an example of a sequential module.
- To write directed testbenches (and a self-checking one for the sequential design) and execute them on the Cadence tool.
- To analyse the simulation logs and waveforms and confirm that the observed behaviour matches the intended design.

2. Tools, Environment and Simulation Flow

Item	Details (taken from the tool logs)
Simulator	Cadence NC-Verilog (ncverilog 64-bit), version 15.20-s086
Compiler / elaborator / engine	ncvlog, ncelab, ncsim (invoked automatically by ncverilog)
Waveform viewer	Cadence SimVision (dump.dsn / dump.trn databases created)
Host	ECE.bmsce.ac.in (Linux, x86-64), Cadence installation under /opt/cadence
Language	Verilog HDL, `` `timescale 1ns/1ps ``
Command used	`ncverilog +access+r <rtl file> <testbench file>`

The `+access+r` option grants read access to all signals so that they can be probed and dumped to the waveform database. A single `ncverilog` command runs the complete flow shown in Fig. 1: *ncvlog* parses and compiles the source files, *ncelab* builds the design hierarchy and snapshot, and *ncsim* executes the snapshot. The testbench writes `dump.vcd`, which is viewed in SimVision.

Fig. 1 Cadence NC-Verilog simulation flow



All stages are launched by one command: ncverilog +access+r <rtl>.v <tb>.v

Project directory structure (as submitted in `1BF24EC028.zip`)

```
1BF24EC028/
 |-- sim/                    Part A : combinational module
 |    |-- rtl/   comparator.v, comparator_4bit.v
 |    |-- tb/    tb_comparator.v, tb_comparator_4bit.v
 |    `-- work/  ncverilog.log, dump.vcd, dump.dsn, dump.trn, INCA_libs/
 `-- sim2/                   Part B : sequential module
      |-- rtl2/  rc.v, ringcounter.v
      |-- tb2/   tb_rc.v, tb_ringcounter.v
      `-- work2/ ncverilog.log, dump.vcd, dump.dsn, dump.trn, INCA_libs/
```

The RTL and testbench files are kept in separate directories, and simulation is always launched from the *work* directory so that all generated databases and logs stay out of the source folders. Files that appear twice (e.g. `comparator.v` and `comparator_4bit.v`) are identical copies of the same module.

3. Part A – Combinational Module: 4-bit Magnitude Comparator

3.1 Theory

A magnitude comparator takes two binary numbers A and B and asserts exactly one of three outputs: $A > B$, $A = B$ or $A < B$. The circuit is combinational – its outputs depend only on the present inputs and there is no clock or memory element. The design here also has three **cascade inputs** (`gt_in, eq_in, lt_in`) so that several 4-bit stages can be chained to compare wider words, exactly like the classic 7485 comparator. When used stand-alone the cascade inputs are tied to `gt_in = 0, eq_in = 1, lt_in = 0`.

3.2 Logic equations

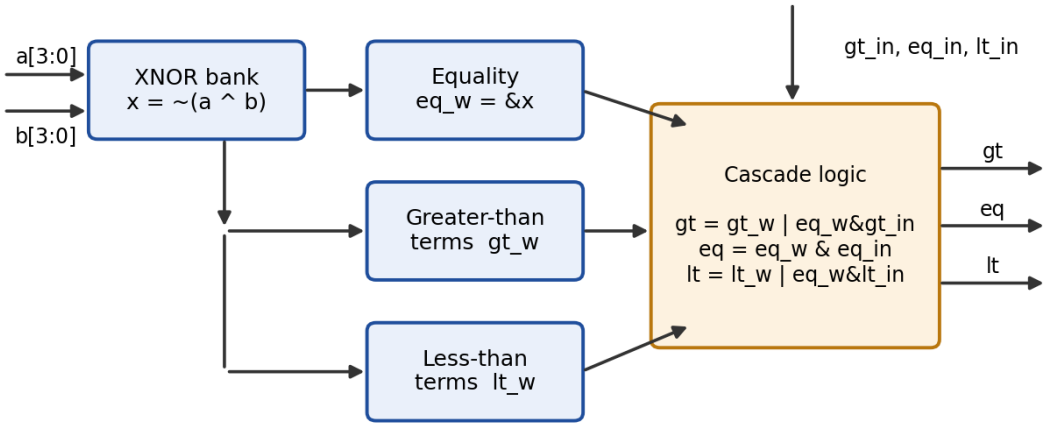
Bit-wise equality is obtained with an XNOR: $\mathbf{x_i = XNOR(a_i, b_i)}$. Using the bits from the MSB downwards:

```
eq_w = x3 . x2 . x1 . x0
gt_w = a3.b3'  +  x3.a2.b2'  +  x3.x2.a1.b1'  +  x3.x2.x1.a0.b0'
lt_w = a3'.b3  +  x3.a2'.b2  +  x3.x2.a1'.b1  +  x3.x2.x1.a0'.b0

gt = gt_w + eq_w . gt_in          eq = eq_w . eq_in          lt = lt_w + eq_w . lt_in
```

The first differing bit, scanning from the MSB, decides the result; if all four bits are equal, the decision is passed on from the cascade inputs.

Fig. 2 Block diagram of the 4-bit cascadable magnitude comparator



3.3 Verilog RTL

Listing 1. comparator_4bit – RTL (sim/rtl/comparator_4bit.v)

```
`timescale 1ns/1ps
// 4-bit cascadable magnitude comparator (stand-alone use: gt_in = 0, eq_in = 1, lt_in = 0)
module comparator_4bit(
    input  [3:0] a, b,
    input        gt_in, eq_in, lt_in,
    output       gt, eq, lt
);
    wire [3:0] x    = ~(a ^ b);
    wire       eq_w = &x;
    wire       gt_w = (a[3] & ~b[3])
                    | (x[3] & a[2] & ~b[2])
                    | (x[3] & x[2] & a[1] & ~b[1])
                    | (x[3] & x[2] & x[1] & a[0] & ~b[0]);
    wire       lt_w = (~a[3] & b[3])
                    | (x[3] & ~a[2] & b[2])
                    | (x[3] & x[2] & ~a[1] & b[1])
                    | (x[3] & x[2] & x[1] & ~a[0] & b[0]);

    assign gt = gt_w | (eq_w & gt_in);
    assign eq = eq_w & eq_in;
    assign lt = lt_w | (eq_w & lt_in);
endmodule
```

The module uses only continuous assignments (`wire` declarations with `assign`) – the hallmark of a combinational description. `x = ~(a ^ b)` is the bit-wise XNOR vector and `&x` is the reduction-AND that forms *eq_w*.

3.4 Testbench

Listing 2. comparator_4bit_tb – directed testbench (sim/tb/tb_comparator_4bit.v)

```
`timescale 1ns/1ps
module comparator_4bit_tb;
    reg  [3:0] a, b;
    wire       gt, eq, lt;

    // cascade inputs tied off for stand-alone use
    comparator_4bit uut (.a(a), .b(b), .gt_in(1'b0), .eq_in(1'b1), .lt_in(1'b0),
                         .gt(gt), .eq(eq), .lt(lt));

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, comparator_4bit_tb);
        $monitor("t=%0d ns  a=%0d b=%0d  ->  gt=%b eq=%b lt=%b", $time, a, b, gt, eq, lt);
        a = 4'd5;  b = 4'd3;   #10;
        a = 4'd3;  b = 4'd5;   #10;
        a = 4'd7;  b = 4'd7;   #10;
        a = 4'd8;  b = 4'd7;   #10;
        a = 4'd0;  b = 4'd15;  #10;
        a = 4'd15; b = 4'd15;  #10;
        $finish;
    end
endmodule
```

Six input pairs are applied, each held for 10 ns, chosen to cover all three outcomes and the extreme values 0 and 15: *greater*, *less*, *equal*, a *boundary greater* case (8 vs 7, where every bit differs), *minimum vs maximum*, and *equal at maximum*. `$monitor` prints a line every time any signal changes, and `$dumpvars` records all signals into `dump.vcd`.

3.5 Execution on the Cadence tool

The design was first compiled on its own (no testbench) to confirm that it was free of syntax errors, and then simulated with the testbench from the *work* directory (compile log shown as an excerpt):

```
cd sim/work
ncverilog +access+r ../rtl/comparator.v ../tb/tb_comparator.v

file: ../rtl/comparator.v
        module worklib.comparator_4bit:v          errors: 0, warnings: 0
file: ../tb/tb_comparator.v
        module worklib.comparator_4bit_tb:v       errors: 0, warnings: 0
Design hierarchy summary:  Modules: 2   Registers: 2   Initial blocks: 1   Cont. assignments: 3
```

Both modules compiled with **0 errors and 0 warnings**. The simulator output (`ncsim> run`) is reproduced below exactly as logged:

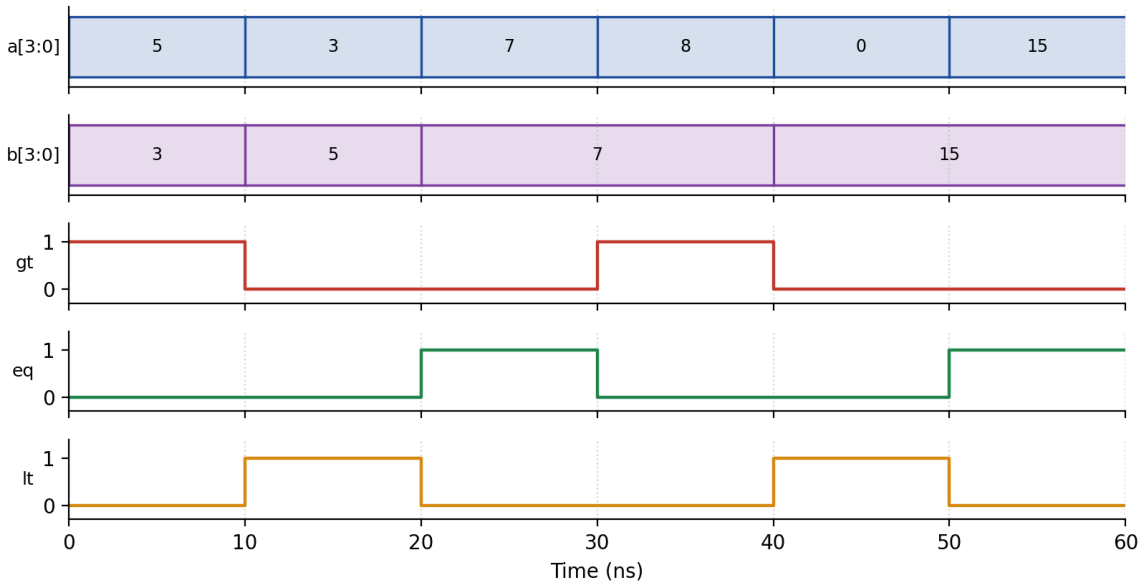
```
t=0 ns  a=5 b=3  ->  gt=1 eq=0 lt=0
t=10 ns  a=3 b=5  ->  gt=0 eq=0 lt=1
t=20 ns  a=7 b=7  ->  gt=0 eq=1 lt=0
t=30 ns  a=8 b=7  ->  gt=1 eq=0 lt=0
t=40 ns  a=0 b=15  ->  gt=0 eq=0 lt=1
t=50 ns  a=15 b=15  ->  gt=0 eq=1 lt=0
Simulation complete via $finish(1) at time 60 NS + 0
```

3.6 Results and verification

Time (ns)	a	b	Binary a / b	gt	eq	lt	Expected	Status
0	5	3	0101 / 0011	1	0	0	A > B	PASS
10	3	5	0011 / 0101	0	0	1	A < B	PASS
20	7	7	0111 / 0111	0	1	0	A = B	PASS
30	8	7	1000 / 0111	1	0	0	A > B	PASS
40	0	15	0000 / 1111	0	0	1	A < B	PASS
50	15	15	1111 / 1111	0	1	0	A = B	PASS

All six vectors produce exactly one asserted output and match the expected relation. The waveform in Fig. 3 (plotted from the `dump.vcd` generated by this run) shows the same behaviour graphically: *gt* is high during 0–10 ns and 30–40 ns, *lt* during 10–20 ns and 40–50 ns, and *eq* during 20–30 ns and 50–60 ns.

Fig. 3 4-bit comparator – waveform plotted from dump.vcd



3.7 Observations

- Outputs change at the same simulation instant as the inputs (no clock), confirming purely combinational behaviour.
- At every time step exactly one of *gt*, *eq*, *lt* is high – the outputs are mutually exclusive and complete.

- At 30 ns (a changes from 7 to 8 with b = 7) the VCD records a zero-width pulse on *lt* before it settles at 0. This is a delta-cycle artefact of zero-delay simulation: *a* updates before the internal XNOR vector *x*, so a stale *x* briefly satisfies one less-than term. It has no duration, and the final settled values are correct. It is not a timing hazard of real hardware, but it illustrates why a gate-level, delay-annotated simulation is still needed for sign-off.

4. Part B – Sequential Module: 4-bit Ring Counter

4.1 Theory

A ring counter is a circular shift register in which the output of the last flip-flop is fed back to the input of the first. A single logic '1' is loaded at reset and then circulates one position per clock pulse, so an N-bit ring counter has **N** distinct states and divides the clock by N. Its outputs are *one-hot*, which makes it useful as a sequencer, a timing/pulse distributor and a simple state-machine encoder with no output decoding. Unlike the comparator, this module is **sequential**: its next state depends on its present state, so it needs memory (flip-flops) and a clock.

Clock pulse	q[3]	q[2]	q[1]	q[0]	q (binary)
Reset	0	0	0	1	0001
1	0	0	1	0	0010
2	0	1	0	0	0100
3	1	0	0	0	1000
4	0	0	0	1	0001 (repeats)

Table: *state sequence of the 4-bit ring counter.*

Fig. 4 Ring counter – circular shift register structure (N = 4)

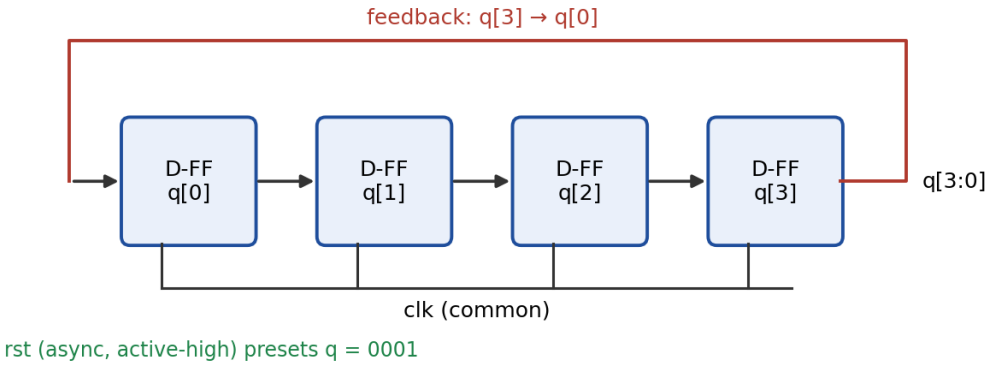
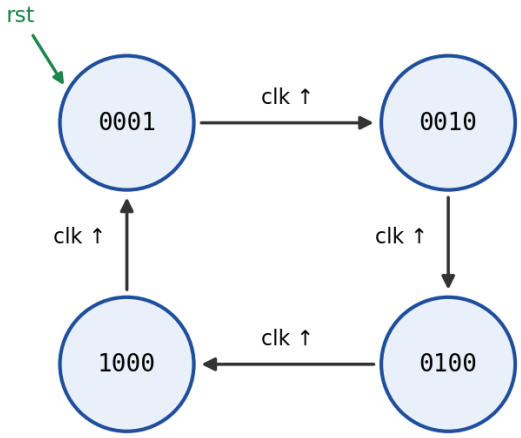


Fig. 5 State diagram of the 4-bit ring counter



4.2 Verilog RTL

Listing 3. ring_counter – RTL (sim2/rtl2/rc.v)

```
`timescale 1ns/1ps

// 4-bit Ring Counter
// Sequence: 0001 -> 0010 -> 0100 -> 1000 -> 0001
// Asynchronous active-high reset

module ring_counter #(parameter N = 4) (
    input clk,
    input rst,
    output reg [N-1:0] q
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= {{(N-1){1'b0}}, 1'b1};
        else
            q <= {q[N-2:0], q[N-1]};
    end

endmodule
```

The module is parameterised by `N` (default 4). The sensitivity list `@(posedge clk or posedge rst)` makes the reset **asynchronous** and active-high: as soon as *rst* rises, *q* is forced to `0001` without waiting for a clock edge. On every other rising clock edge the register is rotated left by one place with `{q[N-2:0], q[N-1]}`, which wraps the MSB around to the LSB. Non-blocking assignments (`<=`) are used, as required for sequential logic.

4.3 Self-checking testbench

Listing 4. tb – self-checking testbench (sim2/tb2/tb_rc.v)

```
`timescale 1ns/1ps

// Self-checking testbench for 4-bit ring counter
// Expected final PASS message:
// RING COUNTER TB : PASSED

module tb;

    reg clk;
    reg rst;
    wire [3:0] q;

    reg [3:0] expected;
    integer i;
    integer errors;

    // Instantiate the design under test
    ring_counter #(.N(4)) uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );

    // Clock period = 10 ns
    initial clk = 1'b0;
    always #5 clk = ~clk;

    // Check the counter after each rising edge
    task check_next_state;
        begin
            @(posedge clk);
            #1;

            expected = {expected[2:0], expected[3]};

            if (q !== expected) begin
                errors = errors + 1;
                $display(
                    "FAIL at %0t: q=%b, expected=%b",
                    $time, q, expected
                );
            end

            // Verify that exactly one bit is HIGH
            if ((q == 4'b0000) ||
                ((q & (q - 4'b0001)) != 4'b0000)) begin
                errors = errors + 1;
                $display(
                    "FAIL at %0t: output is not one-hot, q=%b",
                    $time, q
                );
            end

            $display("%4t ns : q = %b", $time, q);
        end
    endtask

    initial begin
        errors = 0;
        expected = 4'b0001;
        rst = 1'b0;
```

```
        $dumpfile("dump.vcd");
        $dumpvars(0, tb);

        // Apply asynchronous reset
        #2 rst = 1'b1;
        #2;

        if (q !== 4'b0001) begin
            errors = errors + 1;
            $display("FAIL: reset value q=%b", q);
        end
        else begin
            $display("%4t ns : q = %b (RESET)", $time, q);
        end

        // Release reset between clock edges
        #3 rst = 1'b0;

        // Test three complete rotations
        for (i = 0; i < 12; i = i + 1)
            check_next_state;

        // Assert asynchronous reset between clock edges
        #2 rst = 1'b1;
        #1;

        if (q !== 4'b0001) begin
            errors = errors + 1;
            $display("FAIL: asynchronous reset q=%b", q);
        end
        else begin
            $display("%4t ns : q = %b (ASYNC RESET)", $time, q);
        end

        // Release reset and restart the expected sequence
        #2 rst = 1'b0;
        expected = 4'b0001;

        // Verify six additional clock transitions
        for (i = 0; i < 6; i = i + 1)
            check_next_state;

        // Final test result
        if (errors == 0)
            $display("RING COUNTER TB : PASSED");
        else
            $display("RING COUNTER TB : FAILED (%0d errors)", errors);

        $finish;
    end

endmodule
```

Unlike the directed comparator testbench, this one checks itself and reports PASS/FAIL automatically:

- **Clock:** 10 ns period (`always #5 clk = ~clk`).
- **Reset test:** *rst* is asserted at 2 ns, between clock edges, and *q* is checked at 4 ns – proving the reset is asynchronous. Reset is released at 7 ns.
- **Sequence test:** a `check_next_state` task waits for a rising edge, delays 1 ns, rotates a reference value *expected* and compares it with *q* (using `!==` so that X/Z values also fail). It

also verifies that exactly one bit is high (one-hot check). This is run for 12 clock edges = three full rotations.

- **Mid-operation reset:** *rst* is pulsed again at 128 ns, between clock edges, and the counter must return to 0001 immediately. The sequence is then re-checked for 6 more clock edges.
- **Verdict:** an error counter decides whether `RING COUNTER TB : PASSED` or `FAILED` is printed.

4.4 Execution on the Cadence tool

```
cd sim2/work2
ncverilog +access+r ../rtl2/rc.v ../tb2/tb_rc.v

Loading snapshot worklib.tb:v ..................... Done
ncsim> run
4000 ns : q = 0001 (RESET)
16000 ns : q = 0010
26000 ns : q = 0100
36000 ns : q = 1000
46000 ns : q = 0001
56000 ns : q = 0010
66000 ns : q = 0100
76000 ns : q = 1000
86000 ns : q = 0001
96000 ns : q = 0010
106000 ns : q = 0100
116000 ns : q = 1000
126000 ns : q = 0001
129000 ns : q = 0001 (ASYNC RESET)
136000 ns : q = 0010
146000 ns : q = 0100
156000 ns : q = 1000
166000 ns : q = 0001
176000 ns : q = 0010
186000 ns : q = 0100
RING COUNTER TB : PASSED
Simulation complete via $finish(1) at time 186 NS + 0
```

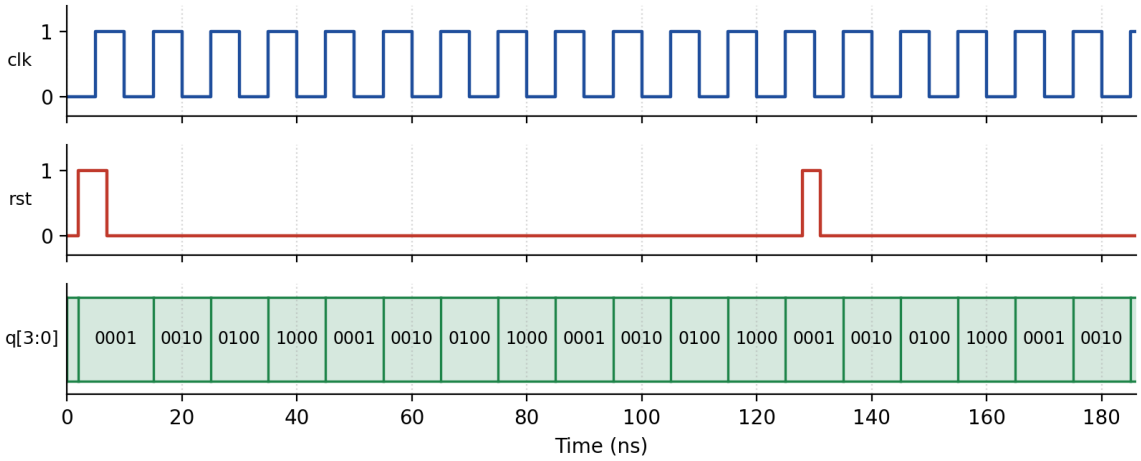
Note on the time stamps. The testbench prints time with `%t`, which displays the value in the simulator's finest unit (1 ps) while the text appended by the testbench says “ns”. The printed numbers are therefore in *picoseconds*: “16000” means 16 ns, which agrees with the “186 NS” reported by `$finish`. The table below converts them to nanoseconds.

4.5 Results and verification

Time (ns)	q observed	Expected	Remark	Status
4	0001	0001	Asynchronous reset	PASS
16	0010	0010	Edge at 15 ns	PASS
26	0100	0100	Edge at 25 ns	PASS
36	1000	1000	Edge at 35 ns	PASS
46	0001	0001	Wrap-around	PASS
56	0010	0010		PASS
66	0100	0100		PASS
76	1000	1000		PASS
86	0001	0001	Wrap-around	PASS
96	0010	0010		PASS
106	0100	0100		PASS
116	1000	1000		PASS
126	0001	0001	Wrap-around (3 rotations done)	PASS
129	0001	0001	Mid-run async reset	PASS
136	0010	0010	Sequence restarted	PASS
146	0100	0100		PASS
156	1000	1000		PASS
166	0001	0001	Wrap-around	PASS
176	0010	0010		PASS
186	0100	0100	Final check	PASS

All 20 observations (2 reset checks and 18 clocked state checks) match the expected one-hot sequence, and the testbench's own verdict is **RING COUNTER TB : PASSED** with zero errors. Fig. 6 shows the waveform generated by this run: *q* advances 0001 -> 0010 -> 0100 -> 1000 -> 0001 on each rising clock edge, and the two reset pulses (2–7 ns and 128–131 ns) force *q* to 0001 at the instant *rst* rises, not at the next clock edge.

Fig. 6 Ring counter – waveform plotted from dump.vcd



4.6 Observations

- Before the first reset *q* is unknown (X in the VCD). It becomes 0001 at 2 ns, when *rst* rises – the counter needs a reset to reach a valid state, as every ring counter does.
- The clock edge at 5 ns has no effect because *rst* is still high; the first counting edge is at 15 ns. This confirms reset has priority over counting.
- The output period is 4 clock cycles = 40 ns, i.e. the clock is divided by 4 and each output bit is high for exactly one clock period.
- The one-hot check never fired, so the counter never entered an illegal state during the run. (A ring counter is not self-correcting: if noise ever produced a non-one-hot pattern it would circulate forever, which is why the reset is important.)

5. Notes on the Tool Logs

Every `ncverilog.log` contains repeated lines of the form

```
ncverilog: *W,DLCVAR (/opt/cadence/installs/IC618/share/cdssetup/cds.lib,5):
           cds.lib Invalid environment variable ''.
```

These are warnings from the lab's shared `cds.lib` setup file, which tries to include `$FOUNDRY/cds.lib` and `$FINFET/cds.lib` while those environment variables are not set on the workstation. They come from the Cadence IC library setup, not from the Verilog design, and they appear identically in every run – the design itself reports **errors: 0, warnings: 0**. The line `*W,RNQUIE: Simulation is complete` in the compile-only logs simply means that, with no testbench, there were no events to execute.

Run (from ncverilog.history)	Files	Outcome
29 Oct 2026, 17:14	comparator.v only	Compiled and elaborated, 0 errors; no stimulus (compile check)
29 Oct 2026, 17:16	comparator.v + tb_comparator.v	6 vectors simulated, all correct
29 Oct 2026, 17:19	ringcounter.v only	Snapshot loaded without errors; no stimulus (compile check)
02 Nov 2026, 14:52	rc.v + tb_rc.v	Self-checking TB: RING COUNTER TB : PASSED

6. Summary and Conclusion

Parameter	Part A – Comparator	Part B – Ring counter
Type	Combinational	Sequential (synchronous, async reset)
Top module / testbench	comparator_4bit / comparator_4bit_tb	ring_counter / tb
Testing style	Directed, 6 vectors, $monitor	Self-checking, 20 checks, PASS/FAIL
Compile result	0 errors, 0 warnings (shown in log)	Snapshot built and run to $finish, no errors
Simulated time	60 ns	186 ns
Outcome	All outputs correct	TB reports PASSED

Both modules were successfully designed in Verilog and executed on the Cadence NC-Verilog tool. The 4-bit comparator correctly flagged *greater*, *equal* and *less* conditions for all test vectors and responded instantly to input changes, as expected of combinational logic. The ring counter produced the correct one-hot rotation 0001 -> 0010 -> 0100 -> 1000 on every clock edge, and its asynchronous reset worked both at start-up and in the middle of operation. The exercise demonstrated the complete simulation flow – compile, elaborate, simulate and waveform inspection – and the practical difference between verifying combinational logic (apply inputs, observe outputs) and sequential logic (clocking, reset behaviour, and state tracking against a reference).

Scope for improvement. The comparator testbench could be extended to an exhaustive sweep of all 256 input combinations with an automatic reference check, and two instances could be cascaded to build an 8-bit comparator using the *gt_in / eq_in / lt_in* ports. The ring counter could be extended with a self-correcting feedback or compared against a Johnson (twisted-ring) counter. Both designs could be taken forward to logic synthesis to study area and timing.